

AFRILANG Stdlib

std/c/mlnormalize

Français. Bibliothèque AFRILANG 'std/c/mlnormalize' (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : normSum, normMean, normMin, normMax, normVar, normStd, normNorm.

'import "std/c/mlnormalize"' · 'use mlnormalize'

- 'normSum(v list number) → number' - 'normMean(v list number) → number' - 'normMin(v list number) → number' - 'normMax(v list number) → number' - 'normVar(v list number) → number' - 'normStd(v list number) → number' - 'normNorm(v list number) → list number' **En-**

glish. AFRILANG library 'std/c/mlnormalize' (tier complex, category complex). Exports 7 function(s): normSum, normMean, normMin, normMax, normVar, normStd, normNorm.

'import "std/c/mlnormalize"' · 'use mlnormalize'

- 'normSum(v list number) → number' - 'normMean(v list number) → number' - 'normMin(v list number) → number' - 'normMax(v list number) → number' - 'normVar(v list number) → number' - 'normStd(v list number) → number' - 'normNorm(v list number) → list number'

Catégorie / Category	Modules complexe (c/)	
Niveau / Tier	complexe	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/mlnormalize' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : normSum, normMean, normMin, normMax, normVar, normStd, normNorm.

```
'import "std/c/mlnormalize"' · 'use mlnormalize'
```

```
- `normSum(v list number) → number`
- `normMean(v list number) → number`
- `normMin(v list number) → number`
- `normMax(v list number) → number`
- `normVar(v list number) → number`
- `normStd(v list number) → number`
- `normNorm(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/mlnormalize"
use mlnormalize
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- normSum(v list number) → number•
normMean(v list number) → number•
normMin(v list number) → number•
normMax(v list number) → number•
normVar(v list number) → number•
normStd(v list number) → number•
normNorm(v list number) → list

4. Exemples d'utilisation

```
import "std/c/mlnormalize"
use mlnormalize
```

```
create result = normSum(/* args */)
say result
```

```
create result = normMean(/* args */)
say result
```

```
create result = normMin(/* args */)
say result
```

```
create result = normMax(/* args */)
say result
```

```
create result = normVar(/* args */)
say result
```

```
create result = normStd(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/mlnormalize"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module mlnormalize

```
function normSum(v list number) returns number
end
```

```
function normMean(v list number) returns number
end
```

```
function normMin(v list number) returns number
end
```

```
function normMax(v list number) returns number
end
```

```
function normVar(v list number) returns number
end
```

```
function normStd(v list number) returns number
end
```

```
function normNorm(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/mlnormalize' (tier complex, category complex). Exports 7 function(s): normSum, normMean, normMin, normMax, normVar, normStd, normNorm.

'import "std/c/mlnormalize"' · 'use mlnormalize'

```
- `normSum(v list number) → number`
- `normMean(v list number) → number`
- `normMin(v list number) → number`
- `normMax(v list number) → number`
- `normVar(v list number) → number`
- `normStd(v list number) → number`
- `normNorm(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/mlnormalize"
use mlnormalize
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- normSum(v list number) → number•
normMean(v list number) → number•
normMin(v list number) → number•
normMax(v list number) → number•
normVar(v list number) → number•
normStd(v list number) → number•
normNorm(v list number) → list

4. Usage examples

```
import "std/c/mlnormalize"
use mlnormalize
```

```
create result = normSum(/* args */)
say result
```

```
create result = normMean(/* args */)
say result
```

```
create result = normMin(/* args */)
say result
```

```
create result = normMax(/* args */)
say result
```

```
create result = normVar(/* args */)
say result
```

```
create result = normStd(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/mlnormalize"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module mlnormalize

```
function normSum(v list number) returns number
end
```

```
function normMean(v list number) returns number
end
```

```
function normMin(v list number) returns number
end
```

```
function normMax(v list number) returns number
end
```

```
function normVar(v list number) returns number
end
```

```
function normStd(v list number) returns number
end
```

```
function normNorm(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/mlnormalize"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/mlnormalize/>

Source (excerpt)

```
module mlnormalize

function normSum(v list number) returns number
end

function normMean(v list number) returns number
end

function normMin(v list number) returns number
end

function normMax(v list number) returns number
end

function normVar(v list number) returns number
end

function normStd(v list number) returns number
end

function normNorm(v list number) returns list number
end
end
```